COMP315 – SEMESTER PROJECT

FINAL PROJECT REPORT

Project Title: Multi-Threaded Inventory and Order Processing System (C++)

Group Name: Code Crusaders

Group Members & Contribution Percentage:

Member Name	% Contribution
Group Leader: Bongekile Dlamini	14.29%
Sheolin Chetty	14.29%
Abiel Michael Govender	14.29%
Braeden Govender	14.29%
Rashiven Govender	14.29%
Antoinette Naidoo	14.29%
Sriman Arjuna Persadh	14.29%

1. Introduction

This report documents the completed implementation of a Multi-Threaded Inventory and Order Processing System developed in C++. It simulates a real-world trading environment where products are managed through a central inventory and customer orders are processed concurrently across multiple threads. The system addresses the core problem of maintaining data integrity and inventory consistency when multiple operations occur simultaneously, which is a challenge commonly faced in modern warehouse and e-commerce environments.

The primary objective of the project was to deliver a fully functional console-based application that demonstrates the practical application of advanced C++ programming concepts. These include object-oriented design, smart pointer memory management, STL container usage, and thread-safe concurrency through mutexes and condition variables.

The scope of the implementation covers all major system operations. Users can add, remove, search, display, and sort products through a menu-driven command-line interface. The system also automates order simulation using a producer-consumer threading model, where producer threads generate orders and worker threads retrieve and process them through a synchronized order queue.

Key features delivered include a flexible product structure that uses polymorphism to support three distinct product types: standard taxable products, discounted products, and products with both a discount and tax applied. The system also includes an inventory manager that is safely controlled using a mutex, an order queue coordinated with condition variables, and a detailed tracking system that records whether each order succeeds or fails. Overall, these features work seamlessly to deliver a system that meets all project requirements.

The solution is built around three main technical ideas. The first is object oriented design, where a base Product class is extended by three derived classes: TaxableProduct, DiscountedProduct, and TaxableDiscountedProduct - each overriding virtual functions to handle their own pricing rules, while allowing all products to be managed through a single, consistent interface. The second is concurrency, where a producer-consumer approach is implemented using seven threads: two producer threads that generate orders and five worker threads that process them - coordinated through a shared OrderQueue using mutexes and condition variables to ensure safe access to shared data. The third is memory safety, where raw pointers are avoided completely by using unique pointers to manage product ownership within both the InventoryManager and the OrderQueue, reducing the risk of memory leaks and invalid references. Together, these elements create a system that is not only functional, but also stable, easy to maintain, and in line with modern C++ development practices.

2. System Overview & Architecture

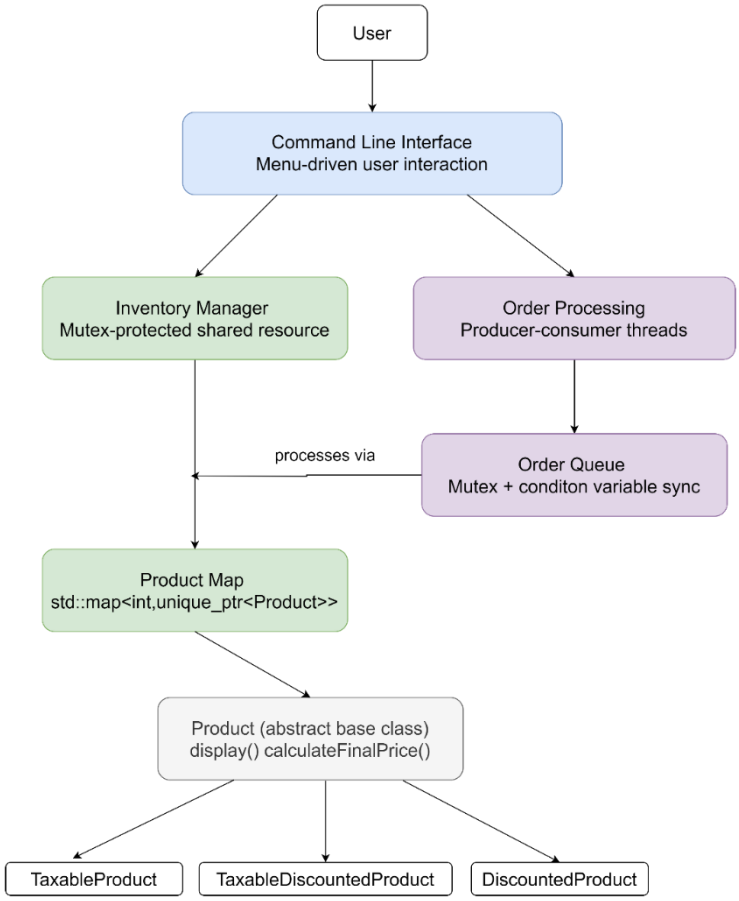

Figure 1: Architecture Diagram

The system follows a modular, object-oriented design aimed at keeping the code maintainable, scalable, and easy to understand. It is implemented as a console-based C++ application, where each component has a clearly defined role, from handling user input to processing orders concurrently. This architecture diagram shows how these components interact during execution.

2.1 Main System Components:

Command Line Interface (CLI):

The CLI is the main point of interaction for the user. It provides a menu-driven interface through showMenu(), allowing actions such as adding, removing, searching, displaying, and sorting products, as well as starting order processing. A do-while loop keeps the system running until exit is selected. User input is passed to handleChoice(), where a switch statement directs the flow of execution. While the CLI temporarily holds product and order data, it does not directly modify the inventory. Instead, it communicates with other components, helping keep responsibilities separate and the system well-structured.

Inventory Manager:

The InventoryManager handles all product storage and operations. Products are stored in a std::map<int, std::unique_ptr<Product>>, ensuring each product has a unique ID and safe memory management through exclusive ownership. Since it is accessed by multiple threads, a mutex is used to protect critical sections and prevent inconsistent data updates.

Order Processing:

The OrderProcessing class manages concurrent order simulation using a producer-consumer model. Producer threads (orderPlacer()) generate orders and adds them to the queue, while worker threads (orderHelper()) retrieve and process them through the InventoryManager. This separation keeps order creation and processing independent. A stop() function allows threads to terminate clearly.

Order Queue:

The OrderQueue stores pending orders as a queue of unique_ptr objects. It uses a mutex and condition variable to handle synchronization between threads. The push() function adds new orders and notifies waiting threads, while pop() retrieves orders. If the queue is empty, threads wait efficiently instead of continuously checking, improving performance.

Product Hierarchy:

The system uses inheritance and polymorphism to support three distinct product types. The base Product class defines shared attributes and three pure virtual functions, display(), calculateFinalPrice(), and getProductType(), that all derived classes must implement. TaxableProduct applies a fixed 15% tax to the base price. DiscountedProduct deducts a configurable discount rate from the base price. TaxableDiscountedProduct applies the discount first and then tax on the reduced amount. Using unique_ptr<Product> allows all three product types to be handled through a single interface, with the correct behaviour determined at runtime through dynamic dispatch.

2.2 Data Flow Between Components:

Data flows in a structured way through the system. The CLI receives user input and forwards requests to the appropriate component. For product-related actions, the CLI interacts directly with the InventoryManager. For order processing, it activates the OrderProcessing component, where producer threads generate orders and worker threads process them through the OrderQueue and InventoryManager. Each order is checked against available stock, and the result is recorded and displayed.

2.3 Concurrency:

Concurrency is introduced during order processing. When the user triggers order simulation, seven threads are spawned - two producer threads running orderPlacer() and five worker threads running orderHelper(). All seven threads run concurrently for three seconds before the shutdown sequence begins. The OrderQueue acts as the shared buffer between order creation and processing, and synchronization is handled using mutexes and condition variables, ensuring safe access and efficient coordination between threads.

2.4 Shared Data Access:

The main shared resource is the product map in the InventoryManager. Since multiple threads may access it, all operations that modify stock are protected using inventoryMutex.

By using std::lock_guard<std::mutex>, access is automatically controlled, ensuring that stock updates happen atomically. This prevents race conditions and keeps inventory data consistent.

3. Class Design & Object-Oriented Structure

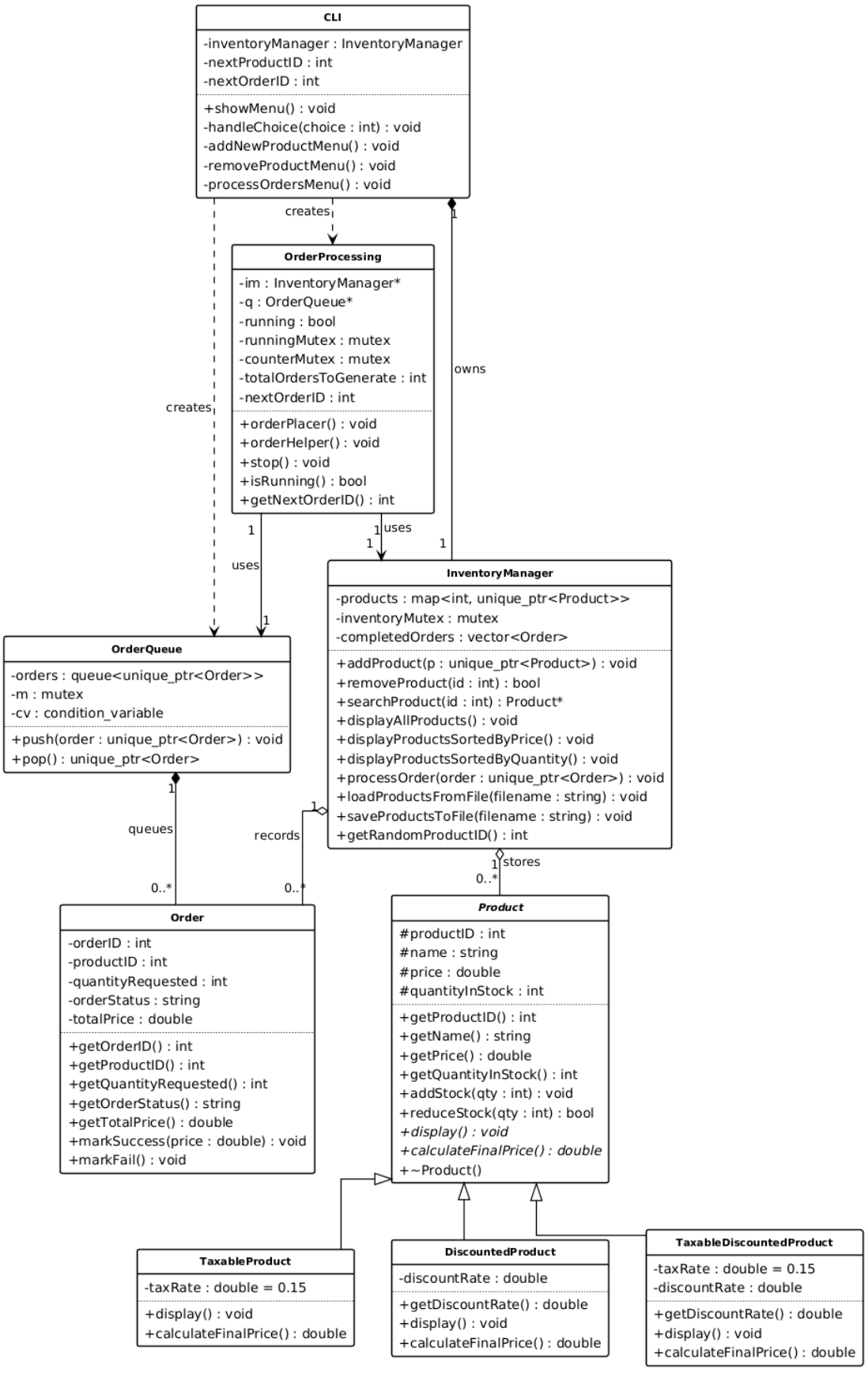

Figure 2: UML Class Diagram Multi-Threaded Inventory and Order Processing System

The system is built around nine classes organised into a clear object-oriented hierarchy. The UML class diagram above shows the full structure and the relationships between components. The subsections on the next page document each class and the OOP principles applied throughout.

3.1 Class Descriptions & Responsibilities

3.1.1 Product (Abstract Base Class):

Product defines the shared attributes and behaviour for all inventory items. Its four core attributes, productID, name, price, and quantityInStock, are declared protected so derived classes can access them directly. The three pure virtual functions, calculateFinalPrice(), display() and getProductType() must be overridden by every concrete subclass. Concrete implementations of reduceStock(), addStock(), and all getters and setters are provided here and shared by all derived types. A virtual destructor ensures correct cleanup when objects are deleted through base class pointers.

3.1.2 TaxableProduct, DiscountedProduct, TaxableDiscountedProduct (Derived Classes):

Each concrete class inherits from Product and overrides calculateFinalPrice() and display() with its own pricing logic. TaxableProduct applies a fixed 15% tax: price × 1.15. DiscountedProduct deducts a configurable discount: price × (1 − discountRate). TaxableDiscountedProduct applies the discount first, then tax on the reduced amount: price × (1 − discountRate) × 1.15. This reflects how pricing is typically handled in real world retail systems, where tax is calculated after discounts are applied. In all three classes, taxRate is declared as a static constexpr double, fixing it at compile time and sharing it across all instances of that type. The discountRate is validated in the constructor, values outside 0.0 to 1.0 are rejected and reset to zero.

3.1.3 InventoryManager:

InventoryManager is the central data component. It stores all products in a std::map<int, std::unique_ptr<Product>>, keyed by product ID. A private std::mutex, inventoryMutex, is locked at the start of every public method using std::lock_guard, ensuring thread safe access across concurrent order processing. The class handles all CRUD operations, sorting via std::sort with lambda comparators, file saving through a CSV formatted Products textfile, and order processing including stock validation and completion recording.

3.1.4 Order:

Order is a data container representing a single customer order. It stores the order ID, product ID, quantity requested, status, and total price. Order status transitions are controlled through two methods: markSuccess(), which records the final price and quantity; and markFail(), which resets price to zero. Order objects move through the system exclusively as std::unique_ptr, with ownership transferring via std::move() at each handoff.

3.1.5 OrderQueue:

OrderQueue is the synchronised middleman between producer and worker threads. It wraps a std::queue<std::unique_ptr<Order>> protected by a std::mutex and a std::condition_variable. The pop() function uses std::unique_lock, rather than lock_guard, because condition_variable::wait() must release and reacquire the lock while a thread blocks. A nullptr is used as a stop signal, with one pushed per worker thread to trigger clean shutdown.

3.1.6 OrderProcessing:

OrderProcessing manages the threading model. Producer threads run orderPlacer(), which generates random orders up to a certain limit and pushes them into the OrderQueue. Worker threads run

orderHelper(), which pops orders and forwards them to InventoryManager::processOrder(). A counterMutex prevents duplicate order IDs when two producers increment the counter simultaneously. The stop() function sets a running flag to false, protected by its own runningMutex.

3.1.7 CLI (Command Line Interface):

CLI owns an InventoryManager object and serves as the single-entry point for user interaction. It presents a numbered menu in a do while loop, validates all input, and routes choices through a switch statement to the appropriate private method. Order processing is triggered by creating a local OrderQueue and OrderProcessing object, spawning threads, waiting three seconds, stopping producers, signalling worker shutdown with nullptrs, and joining all threads before returning.

3.2 Inheritance and the Product Hierarchy:

The product hierarchy is the primary use of inheritance in the system. Product defines everything common to all types, and each derived class extends that foundation with its own pricing logic. All three constructors call the parent constructor through the initialiser list to ensure base attributes are always initialised first. The design follows the open-closed principle, new product types can be added by inheriting from Product and overriding the three virtual functions, with no changes required to InventoryManager, CLI, or OrderProcessing. The final implementation added TaxableDiscountedProduct as a third class beyond the two described in the proposal, because the combination of tax and discount required its own dedicated pricing logic with a specific application order.

3.3 Encapsulation:

All variables in every class is either private or protected. Product attributes are protected to allow direct access within the hierarchy while blocking external access. InventoryManager's product map and mutex are strictly private, no external class can modify inventory directly; all changes go through the public interface. This means any unexpected inventory change can only originate from within InventoryManager itself, significantly reducing the surface area for bugs. Order attributes are private with state changes only possible through markSuccess() and markFail(), preventing random changes of order status or price from outside the class.

3.4 Polymorphism and Virtual Functions:

InventoryManager stores all products as std::unique_ptr<Product>. Because calculateFinalPrice() and display () are virtual, calling either function through a base class pointer at runtime invokes the correct override for the actual object type - whether it is a TaxableProduct, DiscountedProduct, or TaxableDiscountedProduct - with no type checking code required.

Without the virtual keyword, both calls would resolve statically to the base class version, producing identical incorrect behaviour for all product types regardless of their actual type. Declaring the functions pure virtual (= 0) strengthens this further, it prevents Product from being instantiated directly and forces every concrete subclass to provide its own implementation at compile time. If a new derived class omits either override, the compiler rejects it immediately. The virtual destructor ensures that when a derived object is deleted through a Product pointer, the correct destructor chain runs from derived to base, preventing resource leaks in polymorphic deletion.

4. Data Structures & STL Usage

The system makes deliberate use of STL containers and algorithms throughout the implementation to keep things efficient, memory safe, and easy to read. Each container and algorithm is chosen based on the specific needs and access patterns of the component it supports, rather than just using what is most convenient.

4.1 Product Storage:

Products are stored using std::map<int, std::unique_ptr<Product>>, where each product ID maps to its corresponding object. This structure was chosen because it provides efficient lookup, insertion, and deletion, while also automatically enforcing unique product IDs. The map keeps elements sorted by key, which makes displaying products in order straightforward without needing extra sorting. std::unique_ptr is used to ensure that each product has a single owner, the InventoryManager. This means memory is handled automatically when products are removed, avoiding leaks and the need for manual deletion. Storing pointers to the base Product class also allows different product types to be managed through a single interface, with the correct behaviour handled at runtime through polymorphism.

4.2 Order Storage:

Processed orders are stored in std:: vector<Order> completedOrders inside the InventoryManager, where each entry keeps details like the order ID, product ID, quantity, status, and total price. A vector works well here because orders are created and added one after another during runtime. It automatically resizes as needed and allows for efficient iteration when displaying summaries. Since the data is only needed while the program is running, using a simple in-memory structure keeps the design simple without adding unnecessary complexity.

```
class InventoryManager
{
private:

    // Map storing all products in the inventory.
    // Key = Product ID
    // Value = Unique pointer to Product object
    std::map<int, std::unique_ptr<Product>> products;

    // Mutex used to protect inventory from race conditions
    // when multiple threads access the product map
    std::mutex inventoryMutex;

    std::vector<Order> completedOrders;
```

Figure 3: InventoryManager.h - std::map and std::vector declarations

4.3 Order Queue:

Pending orders awaiting processing are managed using std::queue<std:: unique_ptr<Order>> inside the OrderQueue class. A queue is the correct choice here because orders must be processed in the same sequence they are received, following a first-in, first-out order. The use of unique pointers within the queue ensures that each pending order has a single owner at any point in time, and ownership is cleanly transferred from the producer thread to the worker thread when the order is popped from the queue.

The condition variable 'cv' is used alongside the mutex so that the worker threads block efficiently when the queue is empty rather than spinning in a busy-wait loop. When a producer pushes a new order, notify_one() wakes exactly one waiting worker thread to process it.

```
class OrderQueue{
private:
    std::condition_variable cv;
    std::queue<std::unique_ptr<Order>> orders;
    std::mutex m; //Used to lock the OrderQueue for other threads thus preventing races.
public:
    //Functions for adding and removing from "orders"
    std::unique_ptr<Order> pop();
    void push(std::unique_ptr<Order> order);
};
```

Figure 4: OrderQueue.h - std::queue, std::mutex and std::condition_variable declarations

4.4 STL Algorithms - Searching and Sorting:

The system uses efficient map lookup std::map::find to search for products by their ID, which provides fast and direct access to elements. When the user searches for a product by ID, std::map::find traverses the map and returns an iterator to the matching element if one is found.

Sorting by price or quantity is handled using std::sort. Because std::map only maintains order by product ID, products are first copied into a temporary std::vector before sorting on the selected attribute. This keeps storage and sorting logic cleanly separated, leaving the primary map untouched while allowing flexible display ordering.

```
/*
 Searches for a product by its ID.

 Returns a raw pointer to the product if found.
 Returns nullptr if the product does not exist.
*/
Product* InventoryManager::searchProduct(int productID)
{
    std::lock_guard<std::mutex> lock(inventoryMutex);

    auto it = products.find(productID);

    if (it != products.end())
        return it->second.get();

    return nullptr;
}
```

Figure 5: InventoryManager.cpp - searchProduct() using std::find via map lookup

```cpp
std::sort(temp.begin(), temp.end(),
    [](Product* a, Product* b) // comparison function
    {
        return a->calculateFinalPrice() < b->calculateFinalPrice(); // for any two products 'a' and 'b', return true if 'a' should come before 'b'
    });
```

Figure 6: std::sort lambda - sorting products by final price (ascending)

```cpp
std::sort(temp.begin(), temp.end(),
    [](Product* a, Product* b) // comparison function
    {
        return a->getQuantityInStock() < b->getQuantityInStock(); // for any two products 'a' and 'b', return true if 'a' should come before 'b'
    });
```

Figure 7: std::sort lambda - sorting products by quantity in stock (ascending)\

4.5 How STL Improves Safety and Performance:

Using STL containers and algorithms throughout the system brings clear benefits in both safety and performance. Containers handle their own memory management, which removes the need for manual allocation and greatly reduces the risk of memory leaks. On top of that, using smart pointers as values ensures that object lifetimes are automatically managed based on where they are stored.

Standard algorithms like std::sort and std::find_if also make the code easier to read, since they clearly express what the code is trying to do without needing custom implementations. They are already well-tested and optimised, so there is little reason to rewrite the same logic manually. Overall, these choices help create a system that is safer, more reliable, and easier to maintain.

5. Concurrency Design & Thread Safety

5.1 Thread Model:

The system uses a classic producer-consumer pattern. The main thread drives the CLI and when the user triggers order processing, 7 additional threads are spawned consisting of 2 producers orderPlacer () and 5 workers orderHelper(). All 8 threads run concurrently for 3 seconds before the shutdown sequence begins.

5.2 Number of Threads and their roles:

The 2 producer threads each loop, calling getRandomProductID() and rand() to generate random orders, assigning unique IDs under a mutex, then pushing unique_ptr objects into the shared OrderQueue. The 5 worker threads each loop calling pop() on the queue, blocking if it's empty, then forwarding each popped order to InventoryManager::processOrder().

5.3 Shared Resources:

There are four: the InventoryManager::products map, the OrderQueue::orders queue, the running flag, and the pair ordersGenerated/nextOrderID (prevent duplicates).

5.4 Where and why mutexes are used:

inventoryMutex wraps every public InventoryManager method, ensuring the entire read-check-modify sequence on stock is atomic. OrderQueue::m protects the queue for push() and pop(); unique_lock is mandatory here because condition_variable::wait() must temporarily release the lock while a thread sleeps. runningMutex protects the running bool against a torn write during stop(). counterMutex ensures both producers cannot read the same nextOrderID and generate duplicate order numbers.

5.5 How race conditions are prevented:

processOrder() locks inventoryMutex at the top and holds it for the entire function, the stock read, the sufficiency check, and the deduction are one indivisible unit. No two worker threads can ever see the same stock value and both decide to proceed. Without this, negative stock is inevitable.

5.6 How order failures are handled safely:

Inside processOrder(), there are two failure paths - product not found in the map, and stock below the requested quantity - both of which call order->markFail() (setting status to "Fail" and price to 0.0) and return early. Because lock_guard uses RAII, the mutex releases automatically on any return path, including failures. No stock is ever deducted on a failed order.

Evidence of concurrent execution. The output in the widget shows multiple distinct thread IDs processing different orders at the same time. The interleaved [Thread ID: ...] lines confirm true parallel execution, and the stock-level reductions chain correctly across orders on the same product, proving the mutex is working as intended.

6. Memory Management Strategy

The system uses modern C++ memory management to ensure safety and efficiency, especially in a multi-threaded environment. Instead of raw pointers, it relies entirely on smart pointers and the RAII principle, meaning memory is automatically cleaned up when objects go out of scope.

6.1 Where smart pointers are used:

Smart pointers are mainly used in two areas. In the InventoryManager, products are stored using std::unique_ptr, giving the manager full ownership and ensuring products are deleted automatically when removed. In the OrderQueue, orders are also managed with std::unique_ptr, allowing safe transfer of ownership between threads using std::move().

6.2 Difference between shared_ptr and unique_ptr:

Std::unique_ptr allows only one owner at a time and cannot be copied, making it lightweight and safe for clear ownership. Std::shared_ptr allows multiple owners through reference counting but adds some overhead and is used only when shared access is necessary. Each time a new shared pointer is created from an existing one, the reference count is incremented. The object is only destroyed when the reference count reaches zero, meaning all shared owners have released it. Although std:: shared_ptr allows shared access, it is not used in this system because every object has a clearly always defined single owner. Using unique_ptr throughout keeps ownership explicit, avoids the overhead of reference counting, and eliminates any risk of circular references.

6.3 Ownership semantics:

Ownership is clearly defined across the system. The InventoryManager fully owns all products, while orders move step-by-step from producer threads to worker threads, with only one owner at any time. No other component holds a long-lived pointer to any product, the CLI and OrderProcessing components interact with products only through the InventoryManager's public interface, never by holding direct references to the underlying objects. This means that when a product is removed from the inventory, there are no dangling pointers elsewhere in the system that could cause undefined behaviour. Similarly, each order object has a clearly defined ownership chain. The producer thread creates the order and transfers ownership to the OrderQueue via std::move(). The worker thread then takes ownership from the queue when it pops the order for processing. At every stage, exactly one component owns the order, which makes the lifetime of each object straightforward to reason about even in a concurrent execution environment.

6.4 How memory leaks are prevented:

Memory leaks are avoided by using smart pointers exclusively. Objects are automatically destroyed when no longer needed, even if exceptions occur. Since ownership is never shared (no shared_ptr), circular references are eliminated, ensuring all memory is properly released without manual cleanup. Additionally, the use of RAII guarantees that resource deallocation is tied to scope, so memory is freed as soon as objects go out of scope. This approach removes the risk of forgotten delete calls and makes the system more robust and easier to maintain.

7. User Interface Design

The system uses a simple, menu driven command line interface that makes it easy for users to manage inventory and orders. It's designed to be clear and user friendly, with number based inputs to reduce mistakes. Input is validated at every step, and helpful error messages are shown when invalid data is entered.

7.1 CLI Menu Structure:

The main menu is displayed through the showMenu() function and remains active through a do while loop until the user selects the exit option. Each menu option is clearly numbered and labelled, and the handleChoice() function uses a switch statement to route the user's selection to the correct operation. The seven available options cover all core system functionality and are presented as follows:

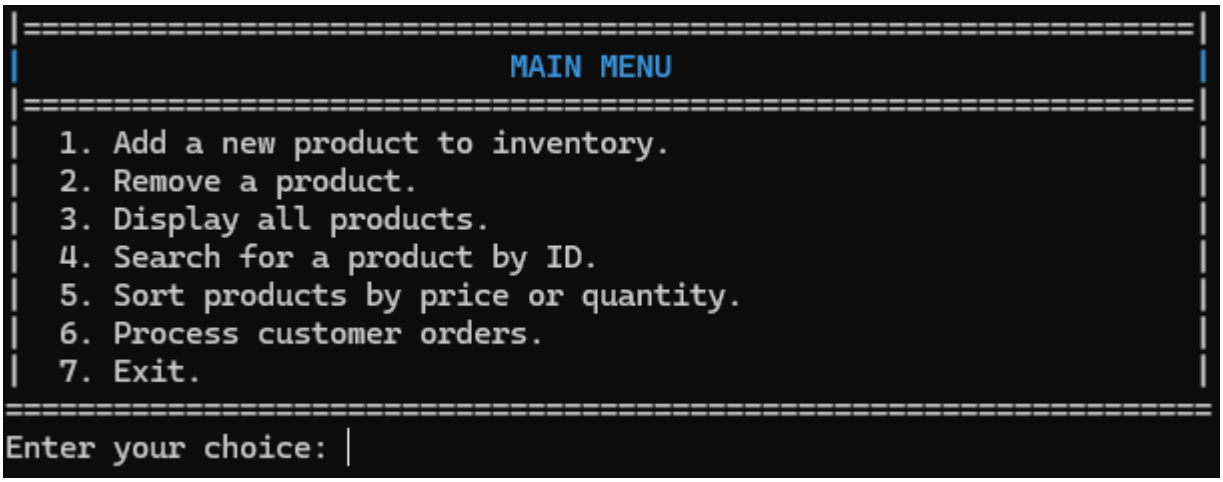

Figure 8: CLI Main Menu

7. 2 User Flow:

Each menu option guides the user through specific inputs or actions. For example, adding a product prompt for its details, while another option starts the multi-threaded order simulation, showing live processing of orders in the console.

When a product is added, the system displays a confirmation message. When viewing all products, it shows key details like ID, name, price, stock, and the final calculated price in a clear and structured format:

DISPLAY ALL PRODUCTS

ID	Type	Name	Price	Qty	Tax	Discount	Final Price
1	Taxable Product	Laptop	R15000.00	2	15	-	R17250.00
2	Discounted Product	Shoes	R1200.00	3	-	20%	R960.00
3	Tax & Discount Product	Phone	R10000.00	2	15%	10%	R10350.00
4	Taxable Product	TV	R8000.00	0	15	-	R9200.00
5	Discounted Product	Chocolate	R25.00	40	-	15%	R21.25
6	Tax & Discount Product	Tablet	R6500.00	1	15%	5%	R7101.25

Figure 9: Displaying All Products

During order processing, the system displays clear output for each thread, including the thread ID, product, quantity, result, and updated stock. This shows that multiple threads are running at the same time and that inventory is being updated correctly:

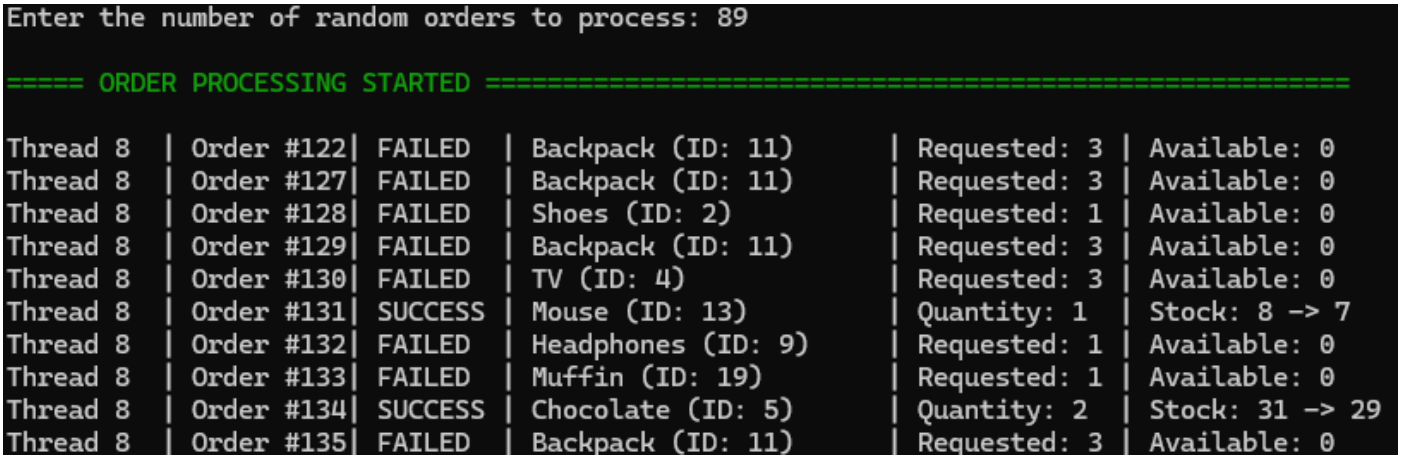

Figure 10: Concurrent Order Processing

The system follows a simple and consistent flow. When the application starts, the main menu appears, and the user selects an option by entering a number. The system processes the request, displays the result, and then returns to the main menu. This continues until the user chooses to exit, at which point all threads are safely stopped and the program closes.

For product management, the user enters details, the system performs an action, and the result is shown. For order processing, the system spawns seven threads, two producers and five workers running concurrently, displaying live, mixed output as orders are handled. Throughout, the system provides clear feedback and error messages, making it easy to use and understand.

8. System Functionality & Testing

The system was tested under normal and edge-case conditions to evaluate inventory operations, concurrent order processing and stock consistency. Overall, the system performed reliably across all scenarios, with no errors observed.

8.1 Normal Operation Tests:

Adding Products:

Products of different types were successfully added through the CLI. The system stored them correctly, rejected duplicate IDs, and displayed accurate details. Final prices were calculated properly, confirming correct runtime polymorphism.

Removing Products:

Products were removed using their ID, and the system confirmed successful deletion. Follow-up searches verified that removed items were no longer in the inventory.

Searching and Sorting:

Search operations returned correct results, and sorting by price and quantity worked as expected using standard algorithms, without affecting the original data.

8.2 Edge Case Tests:

Invalid Input:

Incorrect inputs (e.g. negative values or invalid menu options) were handled gracefully with clear error messages, and the system remained stable :

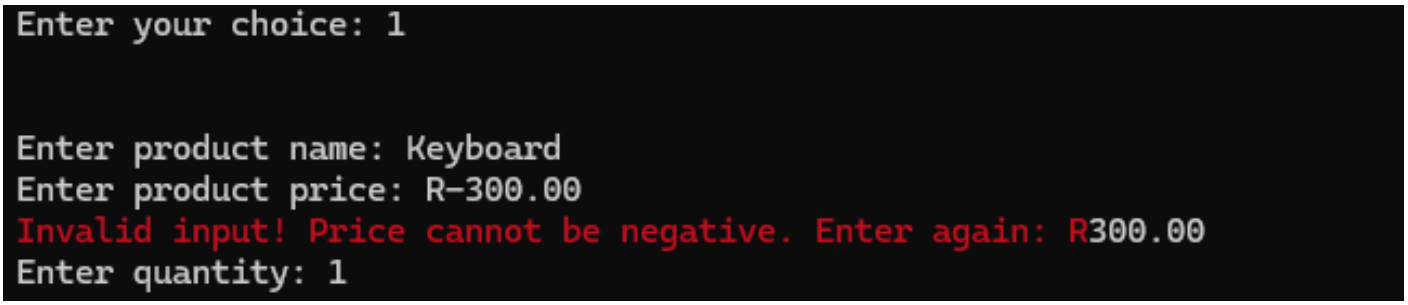

Figure 11: Invalid Input

Insufficient Stock:

Orders exceeding available stock were safely rejected. Stock levels remained unchanged, showing that updates are handled atomically.

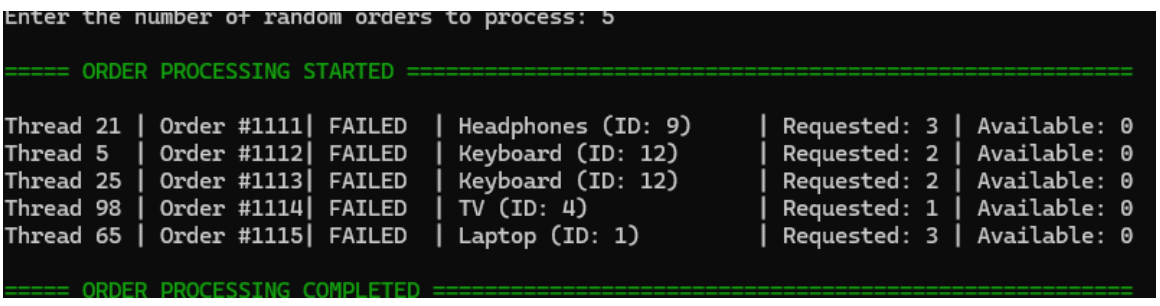

Figure 12: Insufficient Stock

9	Taxable Product	Printer	R3000.00	10	15	-	R3450.00
10	Discounted Product	Backpack	R500.00	15	-	18%	R410.00
11	Tax & Discount Product	Keyboard	R750.00	15	15%	8%	R793.50

Figure 13: Stock Remains Unchanged After Failed Order

8.3 Concurrent Order Tests:

Multiple Threads, Same Product:

Simultaneous orders from seven concurrent threads, two producers and five workers were processed safely using a mutex. Stock levels updated correctly, with no inconsistencies or negative values.

Producer–Consumer Coordination:
The order queue handled concurrent producers and consumers efficiently. Threads synchronized correctly using condition variables, with no deadlocks and all orders processed.

8.4 Large Inventory Test:

The system was tested with 100+ products and handled all operations smoothly. Sorting, displaying, and concurrent processing worked without performance or memory issues, confirming scalability and proper memory management. For the sake of practicality in this report, a total of 6 products has been chosen to demonstrate its sorting capability.

PRODUCTS SORTED BY PRICE

ID	Type	Name	Price	Qty	Tax	Discount	Final Price
15	Taxable Product	Muffin	R5.00	20	15	-	R5.75
5	Discounted Product	Chocolate	R25.00	50	-	15%	R21.25
17	Taxable Product	Banana	R20.00	10	15	-	R23.00

Figure 14: Sorting Products by Ascending Price

8.5 Inventory Consistency:

Across all tests, stock levels were always accurate. Successful orders reduced stock correctly, while failed orders left stock unchanged. This confirms that the system maintains consistency even under concurrent access.

DISPLAY ALL PRODUCTS

ID	Type	Name	Price	Qty	Tax	Discount	Final Price
1	Taxable Product	Laptop	R15000.00	5	15	-	R17250.00
2	Discounted Product	Shoes	R1200.00	20	-	20%	R960.00
3	Tax & Discount Product	Phone	R10000.00	10	15%	10%	R10350.00
4	Taxable Product	TV	R8000.00	15	15	-	R9200.00

Figure 15: Displayed Products Before Orders

```
Enter the number of random orders to process: 7

===== ORDER PROCESSING STARTED ==========================================================

Thread 24 | Order #43 | FAILED  | Tablet (ID: 6)         | Requested: 3 | Available: 2
Thread 24 | Order #48 | SUCCESS | Redbull (ID: 18)       | Quantity: 3  | Stock: 11 -> 8
Thread 24 | Order #49 | SUCCESS | Headphones (ID: 8)     | Quantity: 1  | Stock: 14 -> 13
Thread 40 | Order #46 | SUCCESS | WaterBottle (ID: 13)   | Quantity: 1  | Stock: 94 -> 93
Thread 36 | Order #47 | FAILED  | Banana (ID: 17)        | Requested: 3 | Available: 0
Thread 7  | Order #44 | FAILED  | Banana (ID: 17)        | Requested: 2 | Available: 0
Thread 18 | Order #45 | SUCCESS | Redbull (ID: 18)       | Quantity: 2  | Stock: 8 -> 6

===== ORDER PROCESSING COMPLETED ========================================================
```

Figure 16: Order Processing (Failed & Successful)

ID	Type	Name	Price	Qty	Tax	Discount	Final Price
1	Taxable Product	Laptop	R15000.00	5	15	-	R17250.00
2	Discounted Product	Shoes	R1200.00	20	-	20%	R960.00

Figure 17: Displayed Products After Orders

This demonstrates clean proof of inventory consistency, successful orders reduced stock correctly while failed orders left stock unchanged.

9. Performance and Limitations

The system runs reliably even when multiple threads are working at the same time, as required by the project. During order processing, the mixed console output shows that threads are truly running in parallel, not just one after another. Stock levels are updated correctly across all threads, with no issues like negative inventory or data corruption seen during testing. The use of a mutex in the InventoryManager keeps shared data safe, while the condition variable in the OrderQueue ensures worker threads only run when needed, avoiding wasted CPU usage.

9.1 Observed Performance Under Concurrency:

In practice, the system handles concurrent order processing smoothly within the project's scope. Even with all eight threads running at the same time, orders are created, queued, and processed quickly, with the console clearly showing overlapping activity. While a single mutex is used to protect the inventory, meaning only one thread can update stock at a time, this doesn't noticeably affect performance at this scale. Locks are acquired and released very quickly, and during testing, no threads experienced long waiting times.

9.2 Scalability Considerations:

While the system works well at the current scale, there are some limits we could keep in mind. Using a single mutex for the entire product map means that as more threads are added, they will compete more for access, which could slow things down. In a larger system handling thousands of orders at once, this could become a performance bottleneck. A more scalable solution would be to use finer control, such as giving each product its own mutex instead of locking the whole inventory. Another limitation is that all data is stored in memory, so everything is lost when the program closes. In a real-world system, this would need to be replaced with persistent storage, like a database.

9.3 Known Limitations and Assumptions:

There are a few assumptions that define how the system works. It assumes product IDs are unique integers entered correctly by the user, with basic checks for duplicates but no automatic ID generation. During order simulation, quantities and product choices are randomly generated, so results can differ each time the program runs. The system is also designed for a single-machine environment and does not support distributed or networked use. In addition, the command-line interface cannot be used while order processing is running, it effectively pauses user interaction during threading. This is fine for a simulation, but it would not be suitable for a real production system.

10. Challenges & Solutions

During the development of the multi-threaded inventory system, several technical challenges were encountered that required both collaboration and independent research to resolve.

The most significant challenge was implementing multi-threading and concurrency. Early versions of the system produced inconsistent results that were difficult to trace, which was later identified as a race condition caused by multiple threads accessing shared variables simultaneously. This was resolved by introducing proper synchronization using std::mutex, std::lock_guard, and std::condition_variable, ensuring safe access to shared resources. Structured console outputs were also added to improve debugging and trace execution flow. Understanding how and when to use these tools required extensive reference to resources such as GeeksforGeeks and cppreference.com.

Additional challenges included understanding header file organization, pointer usage (including referencing and dereferencing), and why unique pointers must be moved rather than copied. The group also needed time to become comfortable with shorthand expressions and lambda functions used in STL algorithms.

From a design perspective, the product class hierarchy evolved during development. Initially limited to TaxableProduct and TaxableDiscountedProduct, the structure was expanded to include a standalone DiscountedProduct, improving flexibility and extensibility.

Overall, these challenges strengthened the group's understanding of concurrency, modern C++ memory management, and the importance of adapting system design as requirements evolve.

11. Individual Contribution Summary

Member Name	% Contribution	Description
Group Leader: Bongekile Dlamini	14.29%	Group Leader, Presenter (Phase 1) Class Implementation, CLI improvement and system refinement, code verification and testing, documentation (Phase 1), Demo Video.
Sheolin Chetty	14.29%	Inventory Management Class Implementation, Phase 2 Code Demo Video, code verification and debugging.
Abiel Michael Govender	14.29%	System Design, Order Processing Development, and key contributions to system logic and problem solving. Code verification and debugging
Braeden Govender	14.29%	Presenter (Phase 1), Implemented core classes, CLI interface, system refinement and text file I/O, documentation (Phase 1), code verification and testing.
Rashiven Govender	14.29%	System design, order processing, code verification and debugging, support with STL, documentation.
Antoinette Naidoo	14.29%	Primary author of documentation, system architecture diagram creation, presentation slides (Phase 1), code review.
Sriman Arjuna Persadh	14.29%	Co-author of documentation, presentation slides (Phase 2) diagram creator, code reviewer, and UML/class design, code debugging.

The project was completed collaboratively, with each member contributing to different aspects of design, implementation, and testing. Regular communication and coordination ensured alignment between all components of the system. Some of the collaborative tools we used include WhatsApp and Microsoft Teams for communication and virtual meetings, Git for version control, we also leveraged AI tools to assist with collaboration and documentation tasks.

12. Conclusion

The Multi-Threaded Inventory and Order Processing System was successfully developed as a fully working C++ console application. It meets the main goal of simulating a real-world trading environment, where products are stored in a shared inventory and customer orders are processed at the same time using multiple threads. All required features were implemented, including adding, removing, searching, and sorting products, as well as handling concurrent orders with accurate stock updates and clear order status feedback.

One of the biggest strengths of the system is how it brings together several advanced C++ concepts into a single, well-structured program. The use of a polymorphic product hierarchy allows different types of products to be handled in a consistent way while still supporting their own pricing behaviour. Multi-threading was carefully implemented using a producer-consumer approach with seven threads, along with mutex locks and condition variables, to ensure that shared data remains correct and stable. In addition, memory management was improved through the use of smart pointers, which made the system safer and easier to maintain.

The team gained valuable experience working with concurrency, especially in identifying and fixing race conditions. The project helped us understand the importance of good design, adaptability, and teamwork when solving complex problems. Overall, this project showed that building a reliable system requires careful planning, proper synchronization, and thorough testing, these are all skills that will be useful in future software development work.

13. References

C++ Documentation and Language References:

cppreference.com. (n.d.). *std::mutex*. Available at: https://en.cppreference.com/w/cpp/thread/mutex [Accessed 20 April 2026]

cppreference.com. (n.d.). *std::unique_ptr*. Available at: https://en.cppreference.com/w/cpp/memory/unique_ptr [Accessed 20 April 2026]

cppreference.com. (n.d.). *std::condition_variable*. Available at: https://en.cppreference.com/w/cpp/thread/condition_variable [Accessed 29 April 2026]

Online Learning Resources:

GeeksforGeeks. (2024). *Multithreading in C++*. Available at: https://www.geeksforgeeks.org/multithreading-in-cpp/ [Accessed 1 May 2026]

GeeksforGeeks. (2024). *unique_ptr in C++*. Available at: https://www.geeksforgeeks.org/unique_ptr-in-cpp/ [Accessed 1 May 2026]

GeeksforGeeks. (2024). *Producer-Consumer solution using threads in C++*. Available at: https://www.geeksforgeeks.org/producer-consumer-solution-using-threads-in-cpp/ [Accessed 1 May 2026]